SmartMed: A Consent-Driven, Layered Web Platform for Medication Management and Adherence Monitoring Using React, Spring Boot and MySQL

Aditya Mishra
Enrollment No. 24EARCS010
Dept. of CSE
Arya College of Engg. & I.T.
Jaipur, Rajasthan, India

Aditya Yadav
Enrollment No. 24EARCS011
Dept. of CSE
Arya College of Engg. & I.T.
Jaipur, Rajasthan, India

Badal Verma
Enrollment No. 24EARCS038
Dept. of CSE
Arya College of Engg. & I.T.
Jaipur, Rajasthan, India

Bhawna Sen
Enrollment No. 24EARCS039
Dept. of CSE
Arya College of Engg. & I.T.
Jaipur, Rajasthan, India

Mentor: Mohit Mishra, Professor, Dept. of Computer Science & Engg., Arya College of Engineering & I.T., Jaipur (Affiliated to Rajasthan Technical University, Kota)

***Abstract*—Poor adherence to prescribed medication remains a persistent barrier to effective treatment, yet many people still manage multi-drug routines with paper notes, phone alarms and disconnected applications that neither preserve a dose history nor allow controlled sharing with caregivers and clinicians. This paper presents SmartMed, a web-based medication management and adherence monitoring system that centralises medication records, schedules, dose events and date-filtered adherence analytics, and that supports patient-controlled, read-only monitoring by caregivers and doctors. SmartMed is implemented as a three-tier architecture comprising a React 18/TypeScript frontend, a Java 17/Spring Boot 3 REST backend and a MySQL 8 database whose schema is versioned with Flyway. Security is addressed through stateless JSON Web Token (JWT) authentication, BCrypt password hashing, database-backed role decisions, per-resource ownership checks, a three-state consent model for care relationships (PENDING, ACTIVE, REVOKED) and rate limiting of authentication requests. Data-integrity mechanisms include per-patient transactional locking for dose generation, a uniqueness-based idempotency guard, conditional dose-state transitions and soft deactivation that preserves history. A pluggable provider abstraction supports a medication interaction-checking workflow; the submitted build uses a mock provider and makes no clinical claim. We describe the design, implementation and verification strategy, relate the work to the literature on adherence technologies, and discuss limitations and future work.**

***Index Terms*—medication adherence, mHealth, consent-based monitoring, role-based access control, JSON Web Token, Spring Boot, React, drug interaction checking.**

I. INTRODUCTION

Medication adherence, the extent to which a patient's behaviour in taking medication corresponds to the agreed recommendations of a healthcare provider, is a cornerstone of effective treatment, particularly for chronic conditions that require long-term therapy [1]. The World Health Organization has estimated that, in developed countries, only about half of the patients with chronic diseases take their medicines as prescribed, and that the proportion is likely to be lower in resource-constrained settings [2]. Sub-optimal adherence has been associated with poorer clinical outcomes, avoidable hospitalisation and higher healthcare costs [3], [4].

The practical difficulty of adherence grows with regimen complexity. A patient may need to take several medicines at different times of the day, keep track of changes made at follow-up visits, and report progress to a family caregiver or a physician. In everyday practice these tasks are commonly handled with paper prescriptions, personal calendars, phone alarms or general-purpose reminder applications. Each of these tools addresses part of the problem, but they typically store information in separate places, retain little structured history, and offer no controlled way of sharing selected information with another person.

Mobile and web-based health tools have been proposed to fill this gap, and systematic reviews report generally favourable but heterogeneous evidence for their effect on adherence [5], [6]. However, much of the available tooling is reminder-centric, and where sharing with caregivers is offered it is often coarse-grained: either everything is shared or nothing is. A system that is trusted with health-related records must treat access control, consent and data integrity as first-class design concerns rather than afterthoughts.

This paper presents **SmartMed**, a medication management and adherence monitoring system built with a React/TypeScript frontend, a Java 17/Spring Boot 3 REST backend and a MySQL 8 database. SmartMed allows a patient to maintain medication records and schedules, record dose outcomes, review date-filtered adherence history and analytics, and invite a caregiver or doctor to monitor selected information under a revocable, patient-controlled consent model. SmartMed is an educational software project; it is not a medical device and makes no diagnostic or prescriptive claims.

The main contributions of this work are:

- **A consent-driven access model** in which caregiver and doctor monitoring is read-only, role-matched and conditioned on an ACTIVE, patient-approved relationship that can be revoked at any time, with authority derived from the current database account rather than from client-supplied claims.
- **A history-preserving dose lifecycle** that combines per-patient transactional locking, a uniqueness-based idempotency guard, conditional state transitions that protect terminal states, and soft deactivation that retains historical records.

- **A layered, migration-managed reference implementation** (controller, service and repository layers; Flyway migrations V1 to V4) accompanied by automated integration, security and migration tests.
- **An interaction-checking workflow** built on a provider abstraction with ownership-verified, all-or-nothing request validation, designed so that the absence of data (NO_DATA) is never presented as evidence of safety.
- **An explicit statement of limitations** intended to prevent application-level metrics from being over-interpreted as clinical measurements.

To structure the evaluation, we pose three research questions. **RQ1:** How can patient consent for caregiver and doctor monitoring be expressed as an enforceable, revocable server-side policy? **RQ2:** How can dose records be kept consistent and historically complete under concurrent requests and medication deactivation? **RQ3:** How can an interaction-checking feature be offered without implying a clinical assurance that the system cannot give? Section VIII-G relates the findings to these questions.

The rest of the paper is organised as follows. Section II reviews related literature. Section III states the requirements and design principles. Sections IV, V and VI describe the architecture, the data model and the security and consent model, respectively. Section VII details the implementation, Section VIII presents the verification strategy and evaluation, Section IX discusses limitations and threats to validity, and Section X concludes.

II. LITERATURE REVIEW

A. Non-Adherence: Definitions and Burden

Vrijens *et al.* proposed a taxonomy that decomposes adherence into initiation, implementation and persistence, and argued for consistent terminology in research and practice [7]. The decomposition is relevant to software design. Implementation adherence concerns whether each scheduled dose is taken as intended, which is precisely the information a dose-tracking system can record, whereas persistence concerns whether therapy continues over time and therefore requires longitudinal history. Brown and Bussell reviewed the causes and consequences of non-adherence and highlighted patient-, therapy- and health-system-related factors, including regimen complexity and limited communication between patients and providers [8]. Osterberg and Blaschke likewise observed that adherence is multifactorial and that no single measurement method is perfect, since self-report, pill counts and electronic records each have strengths and weaknesses [1]. These observations motivate two properties of SmartMed: it keeps a durable record of dose events, and it presents the resulting indicators as application metrics rather than as clinical measures.

B. Intervention Strategies

The Cochrane review by Nieuwlaat *et al.* found that many adherence interventions were complex and that, even among the more effective ones, improvements in adherence and in treatment outcomes were modest [9]. Kini and Ho similarly describe interventions at several levels, from regimen simplification to reminders, education and care-team involvement [10]. A common message across these reviews is that no single intervention is universally effective and that multicomponent approaches, combining tracking, feedback and social or professional support, are more plausible than isolated reminders. SmartMed's decision to combine schedules, dose records, analytics and care-team visibility in one workspace follows this reasoning, although the system has not been clinically evaluated and is not itself a clinical intervention.

C. Technology-Based Adherence Tools

Dayer *et al.* discussed how smartphone adherence applications can benefit patients and providers through reminders, tracking and information sharing, while noting the need for further evaluation [11]. Granger and Bosworth described emerging technologies for supporting adherence, ranging from reminder systems to electronic monitoring [12]. Hamine *et al.* systematically reviewed mHealth interventions for chronic disease management and reported generally positive effects on adherence alongside considerable heterogeneity in study design and quality [5]. A content analysis of adherence applications by Ahmed *et al.* reported wide variation in the features offered [6]. Taken together, these studies suggest that technology can support adherence, that the evidence base is uneven, and that functional coverage, particularly with respect to history, analytics and sharing, differs substantially between products.

Importantly, these works are mostly concerned with behavioural and clinical outcomes. Software-engineering properties that determine whether a system can be trusted with sensitive records, such as access-control semantics, data-integrity guarantees under concurrency and automated testing, are generally outside the scope of such reviews. These are the aspects that SmartMed seeks to foreground.

D. Care-Team Involvement, Consent and Access Control

Sharing adherence data with caregivers and doctors can support timely follow-up, but it creates privacy risk if access is broader than the patient intends. Consent is a central principle of contemporary data-protection regimes, including India's Digital Personal Data Protection Act enacted in 2023 [13]. In software terms, consent must be expressed as an enforceable access-control policy. Role-based access control (RBAC), formalised by Sandhu *et al.* [14], assigns permissions to roles and is widely used in enterprise systems. Role membership alone, however, is insufficient for health-record sharing, because holding the caregiver role does not imply the right to view every patient's data. SmartMed therefore combines role checks with resource ownership and with an explicit, patient-controlled relationship between a particular patient and a particular caregiver or doctor. This can be viewed as relationship-based authorisation layered on top of RBAC.

E. Interaction Checking and Clinical Decision Support

Computerised order entry with decision support has been shown to reduce serious medication errors in hospital settings [15], and a systematic review reports improvements in medication-safety measures, although evidence for patient-level outcomes is more limited [16]. Consumer-facing interaction checkers are a lighter-weight analogue, but their

usefulness depends entirely on the quality and coverage of the underlying interaction knowledge. Ayvaz *et al.* showed that drug-drug interaction information is spread over heterogeneous public sources that are far from complete or consistent [17]. Two design lessons follow. A consumer application should not bundle ad hoc interaction data of unknown provenance, and it should separate the user interface from the knowledge source so that a validated, licensed provider can be integrated later. SmartMed adopts exactly this separation through a provider abstraction and ships a mock provider that returns no clinical data.

F. Security Foundations

JSON Web Tokens (JWT), standardised in RFC 7519, provide a compact, signed representation of claims that is suitable for stateless authentication [18]. Provos and Mazières introduced bcrypt, an adaptive password-hashing scheme whose cost parameter can be increased as hardware improves [19]. The OWASP Top 10 lists broken access control, cryptographic failures and identification and authentication failures among the most critical web-application risks [20]. Representational State Transfer (REST), as defined by Fielding, provides the architectural style of the stateless, resource-oriented API used here [21]. SmartMed builds on these foundations, and Section VI maps its controls to the OWASP risk categories.

G. Summary and Research Gap

To position SmartMed, we derive six capability dimensions from the studies above: (1) structured regimen and schedule management, (2) a dose-event history, (3) analytics over that history, (4) controlled sharing with caregivers and clinicians, (5) interaction information, and (6) security and privacy assurances. SmartMed addresses dimensions (1) to (4) and (6) at prototype level and dimension (5) only structurally, through a provider interface without clinical data. Table I summarises the literature themes that motivate each design decision.

TABLE I
SUMMARY OF LITERATURE THEMES AND THEIR IMPLICATIONS

Theme	Works	Implication for SmartMed
Adherence burden	[1]–[4], [7], [8]	Durable, queryable dose history
Intervention design	[9], [10]	Combine tracking, feedback and support
Digital tools	[5], [6], [11], [12]	Uneven features and evidence; avoid clinical claims
Consent and access	[13], [14]	Relationship-based, revocable consent
Interaction CDS	[15]–[17]	Provider abstraction; honest NO_DATA
Security basics	[18]–[21]	JWT, bcrypt, OWASP-aligned controls

The literature supports the importance of adherence support and the plausibility of technology-based approaches, but leaves four practical gaps that SmartMed addresses at prototype level: (i) an integrated, queryable dose history that supports date-range analytics; (ii) fine-grained, revocable consent for caregiver and doctor monitoring; (iii) integrity guarantees for dose records under concurrent requests and deactivation; and (iv) transparent handling of the limitations of interaction checking. We do not claim clinical effectiveness. The contribution is architectural and methodological.

III. REQUIREMENTS AND DESIGN PRINCIPLES

A. Stakeholders

SmartMed recognises four actors. The **patient** owns medication data and controls consent. The **caregiver** and the **doctor** are invited by a patient and, once the invitation is accepted, may read permitted monitoring data. The **system** validates tokens, enforces access rules, generates dose records, maintains notifications and applies database migrations.

B. Functional Requirements

Ten functional requirements (FR-01 to FR-10) were derived from the problem statement and are listed in Table II together with the module that realises each of them (Section VII).

TABLE II
FUNCTIONAL REQUIREMENTS AND REALISING MODULES

ID	Requirement	Module
FR-01	Register a patient account using name, email and password	Auth
FR-02	Authenticate users and return an access token	Auth
FR-03	Return own profile without password fields	Auth
FR-04	Create, view, update and deactivate medications subject to ownership	Medication
FR-05	Maintain schedules and dose records	Schedule, Dose
FR-06	View adherence history and analytics for date ranges	Analytics
FR-07	Create, accept, decline and revoke care relationships	Care team
FR-08	Let connected caregivers and doctors read permitted monitoring data only	Dashboards
FR-09	Request an interaction check for own medications	Interaction
FR-10	List and update recipient-owned notifications	Notifications

C. Non-Functional Requirements

Security: authentication, password hashing, ownership checks and role restrictions. *Reliability:* persistent records and controlled dose-state transitions. *Maintainability:* a layered architecture with separated frontend and backend code. *Usability:* clear navigation and date-oriented summaries. *Compatibility:* modern browsers and standard REST/JSON communication. *Configurability:* secrets, database details, CORS origins and timezone supplied through environment variables.

D. Assumptions and Constraints

Users are assumed to enter accurate medication information and to protect their own credentials. The backend must be configured with a strong JWT secret and correct database settings. The interaction-checking module is informational and is not connected to a production clinical interaction service.

E. Design Principles

Four principles guided the design. (1) *Server authority:* every authorisation decision is taken on the server from the authenticated principal and current database state, never from values supplied by the browser. (2) *Least privilege:* care-team users receive read-only access limited to patients who have approved an active relationship. (3) *History preservation:* operations that remove medications or schedules from active use must not erase dose history. (4) *Honesty about limits:* where

the system cannot provide clinical assurance, it says so rather than implying safety.

IV. SYSTEM ARCHITECTURE

A. Overview

SmartMed follows a three-part runtime topology: a React/TypeScript browser application, a Spring Boot REST backend and a MySQL database. The backend is built on Spring Boot 3 [22] with Spring Security [23] for authentication and authorisation. The frontend uses React 18 [24] and is built with Vite 5 [25]; Chart.js [26] renders chart-based summaries. MySQL 8 [27] provides relational storage, and Flyway [28] manages schema evolution. The frontend sends JSON requests to the API over HTTPS, and a JWT access token accompanies every authenticated request. Fig. 1 shows the logical request path; responses return through the same layers.

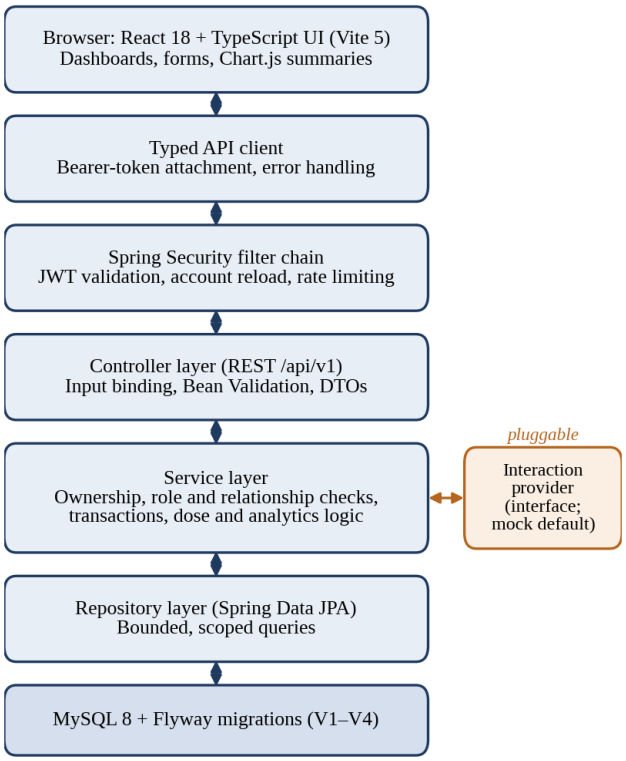


Fig. 1. Logical architecture of SmartMed. Requests pass through the security filter chain, controller, service and repository layers before reaching MySQL; the interaction provider is accessed only through an interface.

B. Layer Responsibilities

The *presentation layer* contains React pages, reusable components, dashboards, forms and date-based views. The *API client* centralises typed requests, token attachment and error handling. *Controllers* expose REST routes, bind and validate input and return response DTOs. The *service layer* implements business rules, transactions, access checks and workflows. *Repositories* perform database reads and writes through Spring Data JPA using bounded queries. The *persistence layer* consists of MySQL tables managed exclusively by Flyway migrations; Hibernate is configured to validate the schema on start-up instead of creating or altering tables.

C. Technology Stack

TABLE III
TECHNOLOGY STACK

Layer	Technologies	Purpose
Frontend	React 18, TypeScript, Vite 5, Chart.js	Component UI, typed API data, builds, charts
Backend	Java 17, Spring Boot 3, Web, Data JPA, Security, Bean Validation, Actuator	REST API, persistence, authN/authZ, validation, health endpoints
Database	MySQL 8, Flyway	Relational storage, versioned migrations
Build/test	Maven wrapper, npm, JUnit, Spring Boot Test, H2	Builds and automated tests

D. API Design

The API is versioned under /api/v1 and follows the REST style [21]. Successful responses use a common envelope containing a success flag, an optional message and a data object; error responses contain a failure flag and a structured error with a code (for example INVALID_CREDENTIALS) and a message. All routes except health, registration and login require a Bearer access token. Table IV lists representative endpoints.

TABLE IV
REPRESENTATIVE API ENDPOINTS

Method	Endpoint	Purpose
GET	/api/v1/health	Public process health response
POST	/api/v1/auth/register	Create a patient account
POST	/api/v1/auth/login	Authenticate an account
GET	/api/v1/users/me	Get current account profile
POST	/api/v1/interactions/check	Informational interaction check
GET	/api/v1/notifications	List recipient notifications
GET	/api/v1/relationships	List relevant care relationships

E. Design Rationale and Alternatives

Several design decisions were made against plausible alternatives. *Stateless tokens versus server sessions:* JWT keeps the API stateless and simplifies hosting the frontend and backend separately, at the price of harder immediate invalidation; SmartMed compensates by reloading the account on each request and comparing the token subject with the database identity, so that authority always reflects current database state. *Relational versus document storage:* the domain is strongly relational, with ownership chains, unique constraints and transactional dose generation, so MySQL with foreign keys and unique constraints was preferred. *Soft versus hard deletion:* adherence history is the core asset of the system, hence soft deactivation and migrations that restrict physical deletion. *Locking and constraints:* a per-patient lock serialises dose generation for one patient without a global bottleneck, while the database constraint acts as a safety net should application logic fail. *Mock provider versus bundled data:* shipping a mock provider avoids embedding clinical content of unverifiable provenance.

F. Deployment and Configuration

The repository describes a deployment option in which the frontend is hosted on Vercel and the Spring Boot backend and MySQL database run on Railway. Production use requires explicit configuration of database connection settings, TLS mode, CORS origins, timezone and a strong JWT secret, all supplied through environment configuration. Spring Boot Actuator provides liveness and readiness endpoints. This documentation is configuration guidance; it does not by itself demonstrate that a production deployment has been tested.

V. DATABASE DESIGN AND DATA INTEGRITY

A. Entities and Relationships

The relational model comprises six core entities: User, Medication, MedicationSchedule, DoseRecord, CareRelationship and Notification (Fig. 2). A user owns many medications; a medication has one or more schedules; a schedule generates many dose records across calendar dates. A patient may hold several care relationships with caregivers or doctors, and each notification belongs to exactly one recipient. Foreign keys and unique constraints enforce referential integrity and support idempotency.

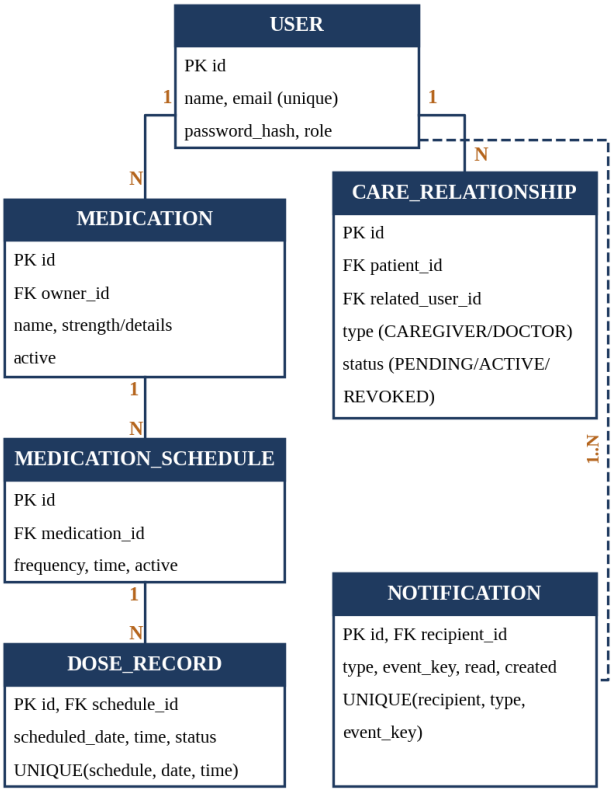


Fig. 2. Entity-relationship summary of the SmartMed schema. Unique constraints on DoseRecord and Notification act as idempotency guards.

B. Schema Migration

Flyway migration V1 creates the initial tables, keys and indexes. Subsequent migrations (V2 to V4) add a medication active flag, support a cancelled-dose state and restrict physical deletion of medications and schedules so that dose history cannot be erased accidentally. Because Hibernate only validates the schema, the migrations are authoritative, and a legacy schema file in the repository is retained merely as a pointer.

C. Dose Generation and State Transitions

Dose generation for a patient executes under a per-patient transaction lock. Existing dose rows for the relevant day are loaded together, and only missing occurrences are created. A uniqueness constraint over (schedule, scheduled date, scheduled time) provides a second, database-level idempotency guard, so that concurrent or repeated generation requests cannot create duplicate doses. State transitions are applied through conditional updates, which protect terminal states from being overwritten when concurrent requests race.

Algorithm 1 summarises the generation procedure for one patient and one calendar day.

Algorithm 1. Idempotent dose generation for patient p on day d

```
1: begin transaction; lock(p)
2: S <- active schedules of p valid on d
3: E <- existing dose rows of p for d
4: for each s in S do
  5: if (s, d, s.time) not in E then
    6: insert dose(s, d, s.time, PENDING)
7: commit  // UNIQUE(schedule, date, time)
        // rejects any duplicate that slips through
```

D. Soft Deactivation and Retention

Deactivating a medication is a soft state change. Its schedules are marked inactive, its pending doses are marked CANCELLED, and its historical dose records are retained. Doses in completed states are never rewritten, and cancelled doses are excluded from adherence analytics so that deactivation does not distort adherence figures.

E. Date and Time Handling

The backend uses an injected clock and a configurable IANA timezone, defaulting to Asia/Kolkata. Date-only values are stored as calendar dates and are not converted through UTC instants, which avoids off-by-one errors around midnight. The frontend timezone must be configured consistently with the backend. Injecting the clock also makes date-dependent logic deterministic in automated tests.

VI. SECURITY AND CONSENT MODEL

A. Authentication

Registration accepts a full name, an email address and a password. The server normalises the email, verifies uniqueness and stores a BCrypt hash [19]. Public registration always creates a PATIENT account, and a role field submitted by a client is ignored, so caregiver and doctor roles cannot be self-assigned. Login compares the submitted password with the stored hash and returns an access token and a user summary only when the credentials are valid; failures return a generic invalid-credential message that does not reveal whether an email address is registered.

B. Token Validation and Client Storage

The JWT subject carries the stable user identifier. For every protected request, the authentication filter validates the

signature and expiry, reloads the current account from the database and checks that the token subject matches the database identity. Authority decisions use the current database account rather than any role value supplied by the browser. On the client, the access token is kept in sessionStorage through a dedicated token-storage utility, which limits its lifetime to the browser session. We note that any script-readable storage is exposed if a cross-site scripting flaw exists; Section IX discusses this trade-off.

C. Layered Authorisation

Authorisation combines three checks. *Role:* the account role must be appropriate for the route. *Ownership:* patient-owned routes derive the user from the authenticated principal, and a medication identifier belonging to another patient is rejected. *Relationship:* care-team routes additionally require an ACTIVE relationship of the matching type (CAREGIVER or DOCTOR) between the caller and the requested patient. Table V summarises the resulting permissions.

TABLE V
ROLES AND PERMISSIONS

Operation	Patient	Caregiver	Doctor
Manage own medications	Yes	No	No
Manage own schedules/doses	Yes	No	No
View own analytics	Yes	No	No
Invite/revoke care relationships	Controls consent	No	No
Read connected patient data	Own data only	Yes, if ACTIVE	Yes, if ACTIVE
Change patient dose status via monitoring	No	No	No
Manage own notifications	Yes	Recipient-scoped	Recipient-scoped

D. Relationship State Model

Consent is represented by a CareRelationship record that moves through the states shown in Fig. 3. A patient sends an invitation, creating a PENDING record. The invitee may accept (ACTIVE) or decline (REVOKED), and the patient may revoke an ACTIVE relationship at any time. Revoked records are retained for traceability, and a later invitation creates a new PENDING record instead of reviving the old one. Because every monitoring request re-checks the relationship, revocation takes effect immediately for subsequent requests.

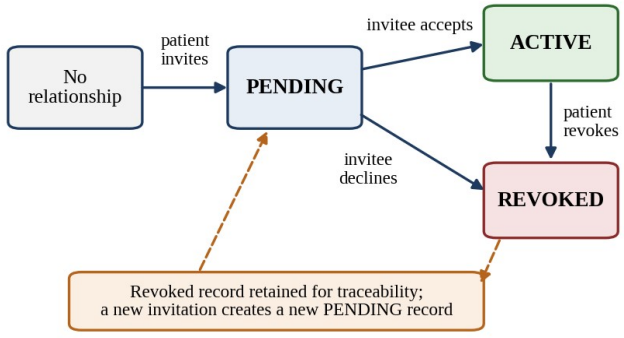


Fig. 3. State model of a care relationship. Only an ACTIVE relationship grants read-only monitoring access.

E. Abuse Resistance and Error Handling

The security package applies rate limiting to repeated authentication failures and to registration requests, which bounds password-guessing and account-creation abuse. The current limiter is local to a single backend instance; a multi-instance deployment would need a shared gateway or distributed store (Section X). A dedicated exception layer converts failures into the consistent error envelope described in Section IV, so that clients can rely on the HTTP status and the response body together.

F. Mapping to OWASP Risk Categories

Table VI relates the implemented controls to the relevant OWASP Top 10 (2021) categories [20]. The mapping is a design-level analysis and does not replace a formal security review.

TABLE VI
CONTROL MAPPING TO OWASP TOP 10 (2021)

Risk category	SmartMed controls
A01 Broken access control	Ownership checks; ACTIVE role-matched relationship for monitoring; recipient-scoped notifications; identity from principal; foreign medication IDs reject the whole request
A02 Cryptographic failures	BCrypt hashes; per-environment random JWT key; explicit database TLS configuration; secrets in protected environment configuration
A03 Injection	Bean Validation on DTOs; parameter binding through Spring Data JPA repositories
A05 Security misconfiguration	Schema validated, not auto-created; configurable CORS allowlist; Flyway-controlled schema
A07 Authentication failures	Signature, expiry and subject checks; generic credential errors; rate limiting

G. Threat Scenarios

We consider four adversaries. (i) *An unauthenticated client* attempting to read protected resources is stopped by the filter chain (TC-04). (ii) *A curious or former care-team member* attempting to read data after consent has been withdrawn fails the relationship check, because access is re-evaluated on every request and is not encoded in the token (TC-06, TC-07). (iii) *A malicious authenticated patient* trying to read or modify another patient's medication by guessing identifiers is stopped by ownership checks (TC-05) and, at the interaction endpoint, by whole-request rejection of foreign identifiers (TC-08). (iv) *A credential-guessing attacker* is slowed by the adaptive cost of BCrypt, generic error messages and rate limiting. Attacks outside this model, such as compromise of the hosting environment, theft of the signing key or script injection into the frontend, are addressed only by operational guidance on secret management and database TLS and by the hardening steps discussed in Section IX.

VII. IMPLEMENTATION

A. Backend Organisation

The backend is organised into eight packages: *controller* (REST mappings), *service* (business rules and transactions), *repository* (persistence access and bounded queries), *entity* (domain objects and enumerations), *dto* (validated request and response contracts), *security* (JWT processing and authentication throttling), *config* (CORS, application clock,

properties and Spring Security) and *exception* (consistent error handling). This separation keeps transport concerns out of business logic and makes each layer testable in isolation.

B. Medication and Schedule Management

Medication endpoints let an authenticated patient create, view, update and deactivate their own records; ownership checks prevent access to another patient's identifiers. A schedule defines when a medication is expected to be taken, and the service layer creates dose records for the relevant dates while preventing duplicate schedule, date and time combinations. Deactivation cascades as described in Section V-D, and inactive schedules cannot generate future pending doses.

C. Dose Tracking and Adherence Analytics

Each dose record captures a scheduled occurrence and its status. Transitions are validated and applied conditionally, and the complete history is retained for later review. The adherence module exposes date-filtered history with pagination (default page size 50, maximum 100) and summary analytics over a date range of up to 366 days. For a range *[t1, t2]*, the adherence percentage is computed from saved dose states as

$$A = 100 \times D_{\text{taken}} / D_{\text{eval}} \qquad (1)$$

where D_{taken} is the number of doses recorded as taken and D_{eval} is the number of doses in the range whose outcome has been recorded, excluding CANCELLED doses. Since the calculation depends entirely on saved events, the quality of the output depends on the completeness and accuracy of what users record. The values are application indicators, not clinical measurements.

D. Care-Team Dashboards

The caregiver and doctor dashboards provide read-only views of connected patients: permitted summaries, saved doses for the current day and adherence analytics. Neither role can alter a patient's dose status through monitoring routes. Every monitoring request follows the path in Fig. 4: the token is validated, the account is reloaded, the service checks for an ACTIVE relationship of the matching role, and only then is a patient-scoped query executed. Invitations are created by exact email lookup of an existing account, which avoids exposing a searchable directory of users.

Algorithm 2 expresses the same decision procedure in pseudocode; note that the role is read from the database account and never from the request.

Algorithm 2. Authorising a care-team monitoring request

```
u <- authenticate(token)
     // signature, expiry, load account
if u = null or token.sub != u.id: deny
if u.role not in {CAREGIVER, DOCTOR}: deny
r <- findRelationship(p, u.id, u.role)
if r = null or r.status != ACTIVE: deny
data <- readOnlyQuery(view, scope = p)
return ok(data)
```

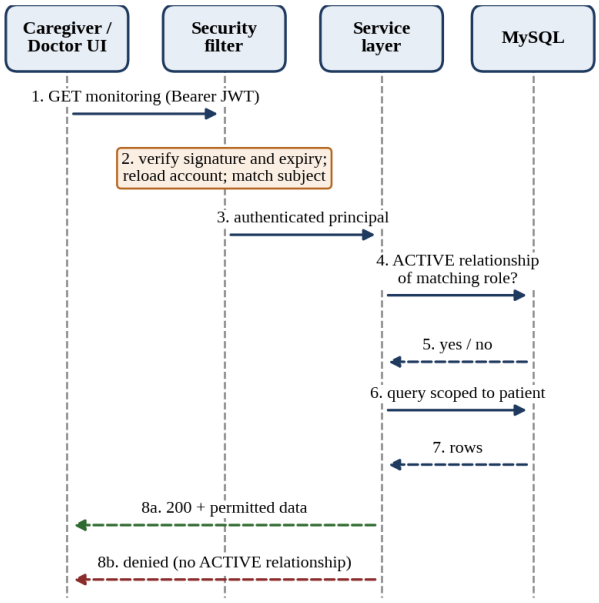


Fig. 4. Sequence of a care-team monitoring request. Access is denied unless an ACTIVE relationship of the matching role exists.

E. Medication Interaction Checker

The patient submits medication identifiers to the interaction endpoint. The backend derives the patient identity from the token and verifies that every medication belongs to that patient before sending the selected medication names to the configured provider. A request containing mixed or foreign identifiers fails as a whole, which prevents the endpoint from being used to probe other patients' records. The provider is accessed through an interface; the default implementation is a mock that returns no clinical data. The result NO_DATA therefore means only that no information was returned and must not be read as evidence that a combination is safe. The user interface states this limitation explicitly.

F. Notification Centre

The notification centre delivers recipient-scoped, in-app notifications for selected events: missed-dose transitions, care-relationship changes and month-to-date adherence attention. The API supports pagination, unread-only filtering, an unread count and read/unread state changes, and all reads and updates are restricted to the recipient. Duplicate notifications are prevented by a recipient, type and event-key combination backed by a database uniqueness constraint. Care-team notifications are sent only to users holding an active matching relationship at the relevant time. The adherence-attention rule is a product rule defined by

$$n \geq 3 \ \wedge \ A_{\text{mtd}} < 70 \qquad (2)$$

where n is the number of recorded doses in the month to date and A_{mtd} is the month-to-date adherence from (1). The 70% threshold is configurable and is not a medically validated measure.

G. Validation and Error Handling

Request payloads are bound to DTOs and validated with Bean Validation before they reach the service layer, so malformed input is rejected at the boundary with a structured

error. History queries are bounded by a maximum page size of 100 and analytics by a 366-day range, which limits response size and database work for any single request. The exception layer maps validation failures, authentication failures, authorisation failures and conflicts to distinct error codes inside the common response envelope, and clients are expected to use the HTTP status and the body together.

H. Frontend Organisation

The frontend separates API modules, reusable components, pages, authentication utilities, date utilities and styling. Components include patient and care-team dashboards, monitoring widgets, notification controls, a care-team panel and the interaction checker. Centralising request behaviour in API functions makes error handling easier to test, and CSS modules together with global style variables provide component-level and shared styling.

I. End-to-End Scenario

A patient registers and logs in, adds two medications with daily schedules and reviews the generated pending doses for the day. After recording outcomes, the patient opens the adherence view for the past month and sees the date-filtered history and the percentage given by (1). The patient then invites a family member by email as a CAREGIVER. Once the invitee accepts, the caregiver dashboard lists the patient and shows the day's saved doses and analytics. When the patient later revokes the relationship, the caregiver's next monitoring request fails the active-relationship check and no data is returned, while the REVOKED record remains for traceability. If month-to-date adherence falls below the threshold with at least three recorded doses, an attention notification is generated, and care-team recipients receive it only while their relationship is active.

VIII. VERIFICATION AND EVALUATION

A. Verification Strategy

Verification targets two questions: does the system satisfy its functional requirements, and are its security boundaries maintained? The repository contains backend integration tests, security-focused tests, Flyway migration tests and a frontend API-client test. Test categories are unit tests (focused utility and business behaviour), integration tests (endpoints and persistence together), authentication and authorisation tests (registration, login, token, role, ownership and relationship rules), migration tests (schema behaviour), frontend API tests (request, response and error handling) and manual acceptance tests of user workflows in a browser.

B. Test Environment

Backend tests use the Spring Boot test framework with JUnit and an in-memory H2 database profile. The suite covers ten functional areas: authentication, medication operations, scheduling and adherence, care relationships, notifications, monitoring, analytics, interaction checks, rate limiting and Flyway migrations. Frontend tests are executed through the npm test script.

C. Specified Test Cases

Table VII lists ten representative cases with their expected outcomes. They are chosen to exercise the consent model, ownership rules and integrity guards described in Sections V and VI.

TABLE VII
REPRESENTATIVE TEST CASES

ID	Scenario	Expected result
TC-01	Register with valid fields	Patient account created; token returned
TC-02	Register duplicate email	Rejected with validation/conflict response
TC-03	Login with wrong password	401 invalid credentials
TC-04	Read profile without token	Request denied
TC-05	Access another patient's medication	Denied by ownership check
TC-06	Caregiver reads without active relationship	Request denied
TC-07	Patient revokes active relationship	Subsequent monitoring access denied
TC-08	Foreign medication ID to interaction checker	Whole request rejected
TC-09	Mark recipient notification read	Only recipient-owned record changes
TC-10	Generate same scheduled dose twice	Duplicate dose prevented

D. Requirements Coverage

Every functional requirement in Table II is realised by an implemented module, and each of the ten tested areas in Section VIII-B corresponds to one or more of those modules. The security-relevant requirements (FR-01 to FR-03, FR-07 to FR-10) are additionally covered by negative test cases such as TC-02 to TC-08, which verify not only that permitted operations succeed but that forbidden ones fail. This coverage is established from the structure of the source code and test suite; we do not report a fresh pass/fail count or code-coverage percentage, and a formal acceptance run (builds, tests, migrations and end-to-end workflows in the target environment) remains part of the deployment checklist.

E. Qualitative Comparison

Table VIII compares SmartMed with the manual and disconnected tools it is intended to complement. The comparison is qualitative and is based on design properties rather than on measured outcomes.

TABLE VIII
QUALITATIVE COMPARISON WITH MANUAL OR DISCONNECTED TOOLS

Feature	Manual/disconnected	SmartMed
Medication records	Separate notes or apps	Centralised per account
Schedule and dose tracking	Manual or reminder-only	Schedules and saved dose states
Adherence review	Manual calculation	Date-filtered history and analytics
Care-team sharing	Informal or broad	Explicit invitation and revocation
Security controls	Varies by tool	JWT, roles and ownership checks
History retention	Notes may be overwritten	Soft deactivation; history kept
Interaction data	Separate resource	Provider interface; mock by default

F. Proposed User Evaluation

Functional verification does not show that the system is usable or that it improves adherence. A suitable next step is a small usability study with representative patients and caregivers, measuring task completion on the core workflows (adding a medication and schedule, recording a dose, inviting and revoking a caregiver) and perceived usability with the System Usability Scale [29]. Any claim about adherence improvement would require a controlled study with appropriate ethical approval.

G. Answering the Research Questions

RQ1 (consent). Consent is enforced by three layers of server-side authorisation (role, ownership and relationship) and by a three-state relationship model that is re-checked on every monitoring request (Sections VI-C and VI-D, Algorithm 2). Cases TC-06 and TC-07 exercise the denial paths, including immediate loss of access after revocation. *RQ2 (integrity).* Consistency is obtained through the per-patient lock, the uniqueness constraint, conditional state updates and soft deactivation (Sections V-C and V-D, Algorithm 1), with TC-10 covering duplicate generation. *RQ3 (honest interaction checking).* The provider abstraction, the all-or-nothing ownership validation, the NO_DATA semantics and the explicit interface disclaimers allow the feature to be exercised without implying clinical assurance (Section VII-E, TC-08). These answers are established at the level of design and test structure; empirical confirmation on MySQL under concurrent load remains future work.

IX. DISCUSSION, LIMITATIONS AND THREATS TO VALIDITY

Clinical scope. SmartMed is an educational project and not a medical device. It does not diagnose, prescribe or issue emergency alerts. The default interaction provider returns no clinical data, so the checker demonstrates the workflow and its safeguards but not clinical value. Likewise, the adherence indicators and the 70% attention rule are product logic, not validated clinical measures.

Data quality. All analytics depend on the user recording doses accurately. Missing or delayed entries lower the apparent adherence, and a user who records nothing cannot be distinguished from one who took no doses. Care-team monitoring is read-only, so a caregiver cannot correct or complete a record on behalf of the patient, which is a deliberate trade-off in favour of integrity and consent.

Client-side token storage. Keeping the token in sessionStorage limits its lifetime to the session but leaves it readable by script. An HttpOnly, SameSite cookie combined with CSRF protection, or short-lived tokens with refresh rotation, would reduce the impact of a cross-site scripting flaw and is a candidate hardening step.

Scalability. Bounded queries and per-patient locking suggest that the cost of a request grows roughly with the number of doses in the requested range and that contention is confined to requests of the same patient. These are design expectations, not measurements: no load test has been performed, and the in-memory rate limiter and single-node assumptions would need to be revisited for a multi-instance deployment.

Operational maturity. Rate limiting is local to a single instance, there is no audit logging or administrative tooling for trusted role provisioning, and in-app notifications are not pushed through reminder channels. No formal security review or privacy impact assessment has been performed.

Threats to validity. Automated tests run against an in-memory H2 database, whose behaviour can differ from MySQL in locking, constraint and date semantics; the concurrency guards in particular should be re-verified on MySQL under load. The evaluation is structural and qualitative, and no user study or performance benchmark is reported. Findings should therefore not be generalised beyond the prototype.

Regulatory and ethical considerations. Handling health-related personal data engages data-protection law, including consent and purpose-limitation obligations [13]. A real-world deployment would need defined retention, access-review, backup and incident-response procedures before accepting real patient data.

X. CONCLUSION AND FUTURE WORK

This paper presented SmartMed, a layered web application for medication organisation and adherence monitoring that combines a typed React frontend, a Spring Boot REST API and a MySQL database. Its distinguishing design choices are a patient-controlled, revocable consent model for read-only care-team monitoring; a history-preserving dose lifecycle with idempotency and concurrency guards; migration-managed schema evolution; and an interaction-checking workflow that is explicit about its limits. A structural verification of requirements, test areas and security controls shows that the core modules are implemented and that the main security boundaries are testable.

Future work includes: (i) integrating a validated, licensed interaction-data provider with documented coverage; (ii) adding configurable reminders and notification channels with explicit user preferences; (iii) conducting accessibility review and usability testing with representative users; (iv) adding audit logging and administrative tooling for trusted role provisioning; (v) improving deployment automation, monitoring, backup and recovery; (vi) evaluating multi-instance rate limiting with a shared gateway or distributed store; (vii) adding localisation and carefully tested timezone preferences; (viii) re-running the integration suite on MySQL and reporting coverage metrics; and (ix) performing a formal security review and privacy impact assessment.

ACKNOWLEDGMENT

The authors thank their mentor, Mohit Mishra, Professor, and Dr. Akhil Pandey, Head of the Department of Computer Science & Engineering, Arya College of Engineering & Information Technology, Jaipur, for their guidance and support during this project.

REFERENCES

[1] L. Osterberg and T. Blaschke, “Adherence to medication,” *N. Engl. J. Med.*, vol. 353, no. 5, pp. 487–497, Aug. 2005.

[2] E. Sabaté, Ed., *Adherence to Long-Term Therapies: Evidence for Action*. Geneva, Switzerland: World Health Organization, 2003.

[3] P. M. Ho, C. L. Bryson, and J. S. Rumsfeld, "Medication adherence: Its importance in cardiovascular outcomes," *Circulation*, vol. 119, no. 23, pp. 3028–3035, Jun. 2009.

[4] M. C. Sokol, K. A. McGuigan, R. R. Verbrugge, and R. S. Epstein, "Impact of medication adherence on hospitalization risk and healthcare cost," *Med. Care*, vol. 43, no. 6, pp. 521–530, Jun. 2005.

[5] S. Hamine, E. Gerth-Guyette, D. Faulx, B. B. Green, and A. S. Ginsburg, "Impact of mHealth chronic disease management on treatment adherence and patient outcomes: A systematic review," *J. Med. Internet Res.*, vol. 17, no. 2, p. e52, Feb. 2015.

[6] I. Ahmed *et al.*, "Medication adherence apps: Review and content analysis," *JMIR mHealth uHealth*, vol. 6, no. 3, p. e62, Mar. 2018.

[7] B. Vrijens *et al.*, "A new taxonomy for describing and defining adherence to medications," *Br. J. Clin. Pharmacol.*, vol. 73, no. 5, pp. 691–705, May 2012.

[8] M. T. Brown and J. K. Bussell, "Medication adherence: WHO cares?" *Mayo Clin. Proc.*, vol. 86, no. 4, pp. 304–314, Apr. 2011.

[9] R. Nieuwlaat *et al.*, "Interventions for enhancing medication adherence," *Cochrane Database Syst. Rev.*, no. 11, Art. no. CD000011, 2014.

[10] V. Kini and P. M. Ho, "Interventions to improve medication adherence: A review," *JAMA*, vol. 320, no. 23, pp. 2461–2473, Dec. 2018.

[11] L. Dayer, S. Heldenbrand, P. Anderson, P. O. Gubbins, and B. C. Martin, "Smartphone medication adherence apps: Potential benefits to patients and providers," *J. Amer. Pharmacists Assoc.*, vol. 53, no. 2, pp. 172–181, Mar./Apr. 2013.

[12] B. Granger and H. Bosworth, "Medication adherence: Emerging use of technology," *Curr. Opin. Cardiol.*, vol. 26, no. 4, pp. 279–287, Jul. 2011.

[13] *The Digital Personal Data Protection Act, 2023*, Act No. 22 of 2023, Government of India, Aug. 2023.

[14] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[15] D. W. Bates *et al.*, "Effect of computerized physician order entry and a team intervention on prevention of serious medication errors," *JAMA*, vol. 280, no. 15, pp. 1311–1316, Oct. 1998.

[16] R. Kaushal, K. G. Shojania, and D. W. Bates, "Effects of computerized physician order entry and clinical decision support systems on medication safety: A systematic review," *Arch. Intern. Med.*, vol. 163, no. 12, pp. 1409–1416, Jun. 2003.

[17] S. Ayvaz *et al.*, "Toward a complete dataset of drug–drug interaction information from publicly available sources," *J. Biomed. Inform.*, vol. 55, pp. 206–217, Jun. 2015.

[18] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.

[19] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf. (FREENIX Track)*, Monterey, CA, USA, Jun. 1999, pp. 81–91.

[20] OWASP Foundation, "OWASP Top 10: 2021." [Online]. Available: https://owasp.org/Top10/

[21] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[22] *Spring Boot Reference Documentation*, 2025. [Online]. Available: https://docs.spring.io/spring-boot/

[23] *Spring Security Reference*, 2025. [Online]. Available: https://docs.spring.io/spring-security/reference/

[24] *React Documentation*, Meta Platforms, Inc., 2024. [Online]. Available: https://react.dev/

[25] *Vite Guide*, 2024. [Online]. Available: https://vite.dev/guide/

[26] *Chart.js Documentation*, 2025. [Online]. Available: https://www.chartjs.org/docs/

[27] *MySQL 8.0 Reference Manual*, Oracle Corp., 2025. [Online]. Available: https://dev.mysql.com/doc/

[28] *Flyway Documentation*, Redgate Software Ltd., 2025. [Online]. Available: https://documentation.red-gate.com/flyway

[29] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.